/

S(1st Sm.)-Comp. Sc.-CSM-105P-Mod. I/Pr./(CBCS)

2018

COMPUTER SCIENCE — PRACTICAL

Paper : CSM-105P

Module-I

(Data Structure)

Full Marks : 50

Instuction : **The examinee must construct flowchart. List all the assumptions, if it is required. Implement your algorithm with maximum number of comments. Test your program with at least 10 test dataset. Output of your program along with the input should be stored in file with naming syntax CSM105P-ROLL NO**

Marks Distribution : Sessional (15), Attendance (5), Experiment (25), Viva-Voce (5)

Answer any ... **One**............Question

1. Write and implement an algorithm that determines whether a stream of parentheses is valid in normal algebraic sense. Choose appropriate data structure(s) for the said purpose, and make clear why and how the data structure(s) is (are) suitable.

2. Write and implement an algorithm that evaluates a given expression in its infix form. Choose appropriate data structure(s) for the said purpose, and make clear why and how the data structure(s) is (are) suitable.

3. Write and implement an algorithm to compute the number of components of an undirected graph. Choose appropriate data structure(s) for the said purpose, and make clear why and how the data structure(s) is (are) suitable.

4. Write and implement an algorithm to differentiate the edges of an undirected graph following the notion of depth-first search, based on your sequence of visiting the vertices. Choose appropriate data structure(s) for the said purpose, and make clear why and how the data structure(s) is (are) suitable.

5. Write and implement an algorithm to differentiate the edges of a directed graph following the notion of depth-first search, based on your sequence of visiting the vertices. Choose appropriate data structure(s) for the said purpose, and make clear why and how the data structure(s) is (are) suitable.

6. Write and implement an algorithm to compute the shortest distance and the path between a pair of vertices of an undirected graph, based on your sequence of visiting the vertices. Choose appropriate data structure(s) for the said purpose, and make clear why and how the data structure(s) is (are) suitable.

7. Write and implement an algorithm to compute the shortest distance and the path between a pair of vertices of a directed graph, based on your sequence of visiting the vertices. Choose appropriate data structure(s) for the said purpose, and make clear why and how the data structure(s) is (are) suitable.

8. Devise a scheme in computing a polynomial C, where C is computed by (i) adding two polynomials A and B, and (ii) subtracting polynomial B from polynomial A. Choose appropriate data structure(s) for the said purpose, and make clear why and how the data structure(s) is (are) suitable.

9. Devise a scheme in computing a polynomial C, where C is computed by (i) multiplying two polynomials A and B, and (ii) differentiating polynomial A. Choose appropriate data structure(s) for the said purpose, and make clear why and how the data structure(s) is (are) suitable.

10. Devise a scheme in representing a sparse matrix A, and transpose this representation of A in lexicographic order. Choose appropriate data structure(s) for the said purpose, and make clear why and how the data structure(s) is (are) suitable.

11. Let X and Y be two lower triangular matrices, each with n rows. Devise and implement a scheme to represent both the triangles in an array Z of size $n \times (n + 1)$. Write algorithms to determine the values of X[i, j] and Y[i, j], $1 \leq i, j \leq n$ from array Z. Choose appropriate data structure(s) for the said purpose, and make clear why and how the data structure(s) is (are) suitable.

12. Write and implement an algorithm that determines the location of a saddle point, if one exists, in a matrix of size $m \times n$. Devise a scheme to make your implementation more efficient. Choose appropriate data structure(s) for the said purpose, and make clear why and how the data structure(s) is (are) suitable.

13. Let A be an $n \times n$ matrix that contains non-zero elements present only in five diagonals, centering and including the major diagonal. If the elements in the band formed by these five diagonals are represented columnwise in an array B, with A[l, 1] being stored in B[l], obtain an algorithm to determine the value of A[i, j], $1 \leq i, j \leq n$ from array B. Choose appropriate data structure(s) for the said purpose, and make clear why and how the data structure(s) is (are) suitable.

14. Consider a sequence of n elements and construct an AVL tree after insertion of each element into the tree. Choose appropriate data structure(s) for the said purpose, and make clear why and how the data structure(s) is (are) suitable.

15. Consider a binary search tree and take necessary steps to make it an AVL tree, if needed. Perform deletion of elements from the AVL tree and make the tree AVL after deletion of each element from the tree. Choose appropriate data structure(s) for the said purpose, and make clear why and how the data structure(s) is (are) suitable.

16. Consider a sequence S of $n \geq 15$ elements and implement *HeapSort* of S. Choose appropriate data structure(s) for the said purpose, and make clear why and how the data structure(s) is (are) suitable.

17. Consider a sequence S of $n \geq 15$ elements and implement *ShellSort* of S. Choose appropriate data structure(s) for the said purpose, and make clear why and how the data structure(s) is (are) suitable.

18. Consider a sequence S of $n \geq 15$ elements and implement *SieveSort* of S. Choose appropriate data structure(s) for the said purpose, and make clear why and how the data structure(s) is (are) suitable.

19. Consider the problem of implementing n-digit binary integers and compute amortized cost, a_i of the problem in terms of n. Choose appropriate data structure(s) for the said purpose, and make clear why and how the data structure(s) is (are) suitable.

———